MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI

Department of Computer Engineering

Academic Year: 2025-2026

FSJP (2113611) Mini Project

Report on

"SkillBarter"

Submitted by:

Adit Halde (337)

Kartik Hulmukh (338)

Vivek Humbe (339)

Submitted to:

Prof.Kaajal Sharma

MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI

Certificate

Department of Computer Engineering

This is to certify that

Adit Halde (337)

Kartik Hulmukh (338)

Vivek Humbe (339)

Have satisfactorily completed this mini project entitled Skill-Barter.
Towards the partial fulfilment of the

BACHELOR OF ENGINEERING IIIrd Semester in

(COMPUTER ENGINEERING) as laid by University of Mumbai

Abstract

SkillBarter is a modern web-based skill exchange platform that enables users to share knowledge and learn new skills through a token-based economy system. The platform facilitates skill exchanges where users can either pay with tokens or offer their own skills in return for learning opportunities. Built using React 19, TypeScript, and modern UI components, the application provides a seamless user experience with features including user authentication, skill marketplace, exchange management, and comprehensive user profiles.
The platform addresses the growing need for peer-to-peer learning and skill development in today's knowledge economy. Users start with 100 welcome tokens and can earn more by teaching skills to others, creating a sustainable ecosystem of knowledge sharing.

Key Features:

- Token-based economy system
- Skill-for-skill exchange mechanism
- User authentication and profile management
- Skills marketplace with search and filtering
- Exchange request and management system
- Responsive design for all devices
- Real-time state management

Problem Definition

Current Challenges in Skill Learning:

1.High Cost of Professional Training: Traditional skill learning platforms charge high fees, making quality education inaccessible to many.
2.Limited Peer-to-Peer Learning: Existing platforms focus on instructor-led courses rather than community-driven knowledge sharing.
3.Lack of Fair Exchange System: No standardized way to exchange skills without monetary transactions.
4.Fragmented Learning Communities: Skills are taught in isolation without building a connected learning community.
5.No Recognition for Teaching: People with expertise have limited opportunities to monetize their knowledge or get recognized for teaching.

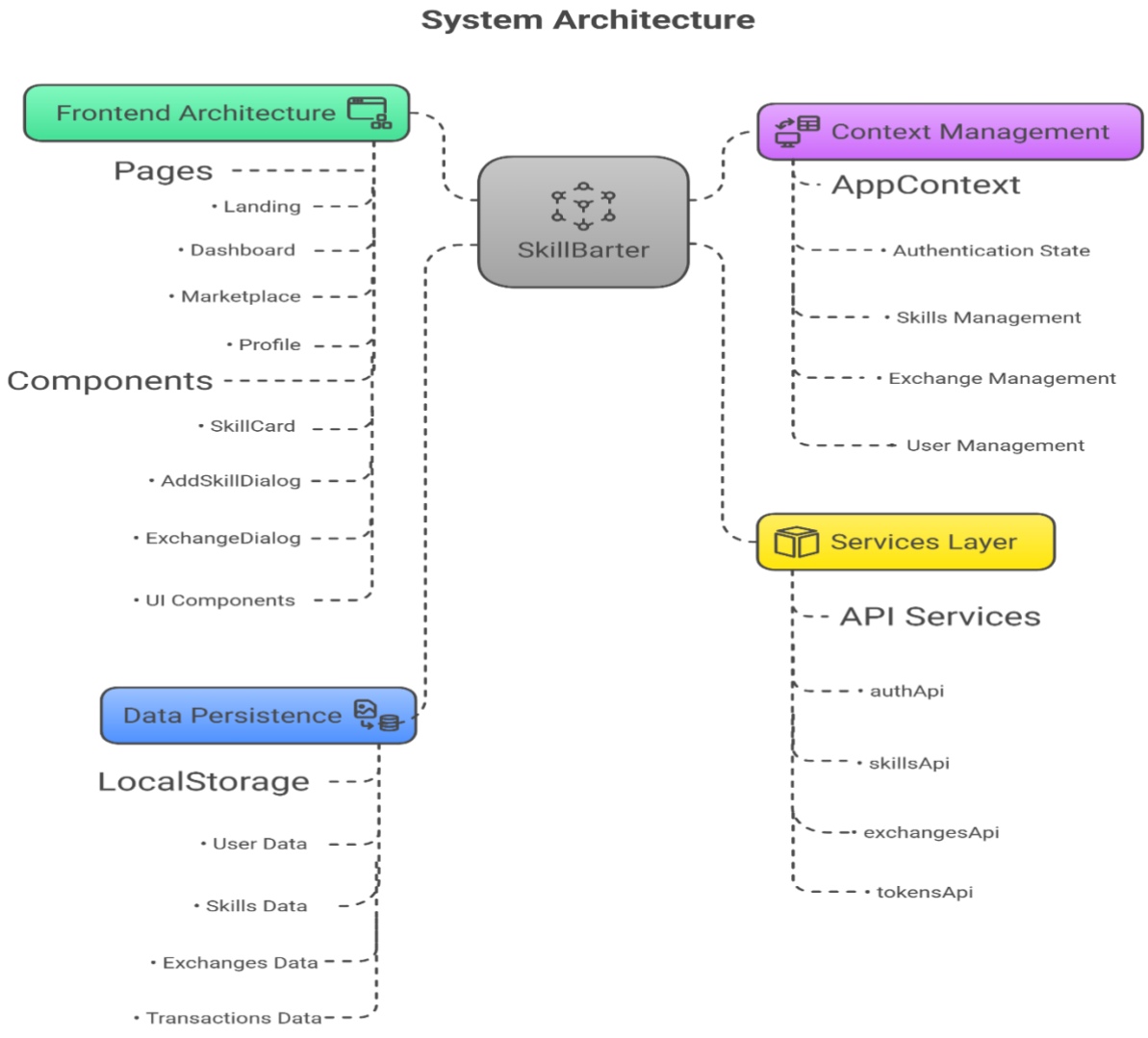
Proposed Solution:

SkillBarter addresses these challenges by creating a comprehensive platform where:

- Users can exchange skills without traditional monetary • Users can exchange skills without traditional monetary transactions
- A token economy ensures fair value exchange
- Community-driven learning promotes peer-to-peer education
- Both learners and teachers benefit from the ecosystem
- Skills are organized in categories for easy discovery

Flow chart /System architecture:

1. Frontend architecture:



Made with Napkin

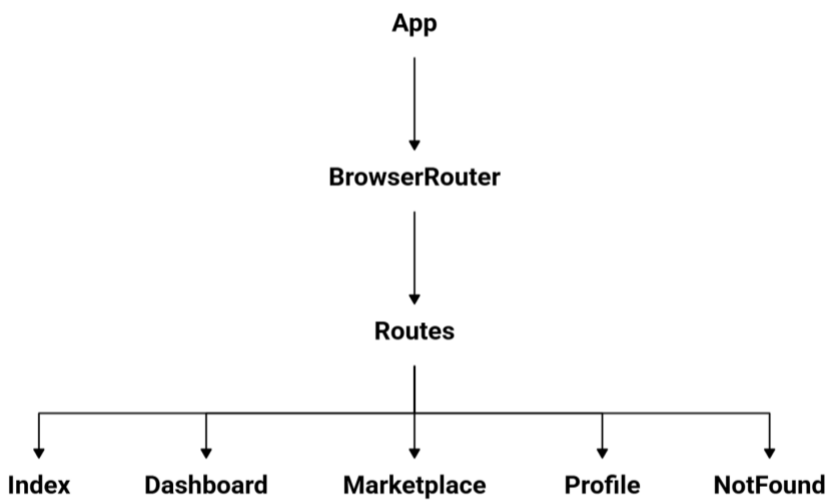
Data Flow Architecture:

User Interaction → Component → Context → API Service → LocalStorag ↓

User Interface ← Component ← Context ← API Response ← Data Retrieval

---

Component Hierarchy:



Hardware & Software Requirements

Hardware Requirements:

Minimum:

- Processor: Intel Core i3 or equivalent
- RAM: 4GB
- Storage: 1GB free space
- Network: Broadband internet connection

Recommended:

- Processor: Intel Core i5 or higher
- RAM: 8GB or more
- Storage: 2GB free space
- Network: High-speed internet connection

Software Requirements:

Development Environment:

• Operating System: Windows 10/11, macOS 10.15+, or Linux Ubuntu 18.04+
• Package Manager: pnpm, npm, or yarn
• Code Editor: Visual Studio Code (recommended)
• Browser: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

Runtime Dependencies:

• React: 19.x
• TypeScript: 5.x
• Vite: 5.x
• Tailwind CSS: 3.x
• Radix UI: Latest version
• Lucide React: Latest version

Development Dependencies:

• ESLint: Code linting
• Prettier: Code formatting
• PostCSS: CSS processing

Browser Compatibility:

• Chrome 90+
• Firefox 88+
• Safari 14+
• Edge 90+
• Mobile browsers (iOS Safari, Chrome Mobile)

Source Code

Project Structure:

```
skill-barter/
├── public/
│   ├── favicon.svg
│   └── robots.txt
├── src/
│   ├── components/
│   │   ├── ui/
│   │   │   ├── button.tsx
│   │   │   ├── card.tsx
│   │   │   ├── dialog.tsx
│   │   │   ├── input.tsx
│   │   │   ├── radio-group.tsx
│   │   │   └── [other UI components]
│   │   ├── AddSkillDialog.tsx
│   │   ├── ExchangeDialog.tsx
│   │   └── SkillCard.tsx
│   ├── context/
│   │   └── AppContext.tsx
│   ├── pages/
│   │   ├── Dashboard.tsx
│   │   ├── Index.tsx
│   │   ├── Marketplace.tsx
│   │   ├── NotFound.tsx
│   │   └── Profile.tsx
│   ├── services/
│   │   └── api.ts
│   ├── types/
│   │   └── index.ts
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
├── package.json
├── tsconfig.json
├── tailwind.config.ts
├── vite.config.ts
└── README.md
```

Key Features Implementation:

1. Token Economy System:

- Users start with 100 welcome tokens
- Earn tokens by teaching skills
- Spend tokens to learn skills
- Transaction history tracking

2. Skill Exchange Mechanism:

- Traditional token payment
- Skill-for-skill exchange
- Exchange status management
- Rating and review system

3. User Authentication:

- Registration with email validation
- Login with session management
- Profile customization
- Secure logout

4. Skills Management:

- Add new skills with categories
- Edit existing skills

- Skill marketplace browsing
- Search and filter functionality

Key UI/UX Features:

- **Modern Design**: Purple/indigo gradient color scheme
- **Responsive Layout**: Mobile-first design approach
- **Interactive Elements**: Smooth animations and hover effects
- **Accessibility**: Proper contrast ratios and semantic markup
- **User Feedback**: Loading states, error messages, success notifications

---

1. **Frontend Dependencies (package.json)**

```json
{
  "name": "frontend",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "react": "^19.1.1",
    "react-dom": "^19.1.1",
    "react-router-dom": "^7.7.1",
    "@reduxjs/toolkit": "^2.8.2",
    "axios": "^1.11.0",
    "tailwindcss": "^4.1.11",
    "framer-motion": "^12.23.12",
    "react-hook-form": "^7.62.0",
    "yup": "^1.7.0"
  }
}
```

Backend Dependencies (pom.xml)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.0</version>
    </parent>

    <groupId>com.skillbarter</groupId>
    <artifactId>skill-barter-backend</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>${jwt.version}</version>
        </dependency>
    </dependencies>
</project>
```

2. Backend Core Application

Main Application Class

```java
package com.skillbarter.app;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@SpringBootApplication
@EnableJpaAuditing
public class SkillBarterApplication {
    public static void main(String[] args) {
        SpringApplication.run(SkillBarterApplication.class, args);
    }
}
```

User Entity (Core Business Model)

```java
@Data
@Entity
@Table(name = "users")
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false, unique = true)
    private String email;

    private String firstName;
    private String lastName;
    private String profilePictureUrl;
    private String bio;

    @ElementCollection(fetch = FetchType.EAGER)
```

```java
    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "user_roles", joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "role")
    private Set<String> roles = new HashSet<>();

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    // UserDetails implementation
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        Set<GrantedAuthority> authorities = new HashSet<>();
        for (String role : roles) {
            authorities.add(new SimpleGrantedAuthority("ROLE_" + role));
        }
        return authorities;
    }
}
```

Skill Entity (Core Business Logic)

```java
@Data
@Entity
@Table(name = "skills")
public class Skill {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @Column(length = 1000)
    private String description;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "category_id", nullable = false)
    private SkillCategory category;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private SkillLevel level;

    @OneToMany(mappedBy = "skill", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
```

```java
    @OneToMany(mappedBy = "skill", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<SkillOffer> offers = new ArrayList<>();

    @OneToMany(mappedBy = "skill", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<SkillRequest> requests = new ArrayList<>();

    public enum SkillLevel {
        BEGINNER, INTERMEDIATE, ADVANCED, EXPERT
    }
}
```

3. Frontend Application Structure

Main App Component with Routing

```tsx
function App() {
  return (
    <Router>
      <ThemeProvider>
        <LanguageProvider>
          <AuthProvider>
            <TokenProvider>
              <Suspense fallback={<LoadingSpinner size="lg" />}>
                <Routes>
                  {/* Public routes */}
                  <Route element={<PublicRoute />}>
                    <Route path="/login" element={<Login />} />
                    <Route path="/register" element={<Register />} />
                  </Route>

                  {/* Protected routes */}
                  <Route element={
                    <ProtectedRoute>
                      <Layout>
                        <Outlet />
                      </Layout>
                    </ProtectedRoute>
```

```
                    </ProtectedRoute>
                  }>
                    <Route index element={<Home />} />
                    <Route path="dashboard" element={
                      <RoleBasedRoute allowedRoles={['user', 'admin']}>
                        <DashboardPage />
                      </RoleBasedRoute>
                    } />
                    <Route path="exchange" element={<SkillExchange />} />
                  </Route>

                  {/* Special routes */}
                  <Route path="/unauthorized" element={<Unauthorized />} />
                  <Route path="/404" element={<NotFound />} />
                  <Route path="*" element={<Navigate to="/404" replace />} />
                </Routes>
              </Suspense>
            </TokenProvider>
          </AuthProvider>
        </LanguageProvider>
      </ThemeProvider>
    </Router>
  );
}
```

4. Key Features Implementation

```java
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody AuthRequest request) {
        // Authentication logic with JWT token generation
        String token = jwtService.generateToken(user);
        return ResponseEntity.ok(new AuthResponse(token, userDTO));
    }

    @PostMapping("/register")
    public ResponseEntity<UserDTO> register(@RequestBody @Valid RegisterRequest request) {
        // User registration with password encryption
        User user = userService.createUser(request);
        return ResponseEntity.ok(userMapper.toDTO(user));
    }
}
```

Home Page (Frontend Main Interface)

```
const Home: React.FC = () => {
  const features = [
    {
      icon: <UserPlusIcon className="h-8 w-8 text-primary-500" />,
      title: 'Create Your Profile',
      description: 'Showcase your skills, experience, and what you can offer to the community.',
    },
    {
      icon: <ArrowPathIcon className="h-8 w-8 text-secondary-500" />,
      title: 'Find Matches',
      description: 'Our AI-powered matching system connects you with people who need your skills and have skills you need
    },
    {
      icon: <ChatBubbleLeftRightIcon className="h-8 w-8 text-emerald-500" />,
      title: 'Start Exchanging',
      description: 'Connect, chat, and arrange skill exchanges that benefit everyone involved.',
    },
  ];

  return (
    <div className="min-h-screen">
      <Hero />
      <SkillsCarousel />
      <SmartMatch />
      {/* Features section with animations */}
    </div>
  );
};
```

5. Deployment Configuration

Docker Compose (Full-Stack Deployment)

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: skill-barter-postgres
    environment:
      POSTGRES_DB: skill_barter
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build: ./backend
    container_name: skill-barter-backend
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/skill_barter
    depends_on:
```

```yaml
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/skill_barter
    depends_on:
      - postgres

  frontend:
    build: ./frontend
    container_name: skill-barter-frontend
    ports:
      - "3000:3000"
    environment:
      REACT_APP_API_URL: http://backend:8080/api
```

6. Token Economy System

Token Transaction Model

```
@Data
@Entity
@Table(name = "token_transactions")
public class TokenTransaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "from_user_id")
    private User fromUser;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "to_user_id")
    private User toUser;

    @Column(nullable = false)
    private Integer amount;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
```

```
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TransactionType type;

    @Column(length = 500)
    private String description;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime createdAt;

    public enum TransactionType {
        EXCHANGE, REWARD, PURCHASE, REFUND
    }
}
```

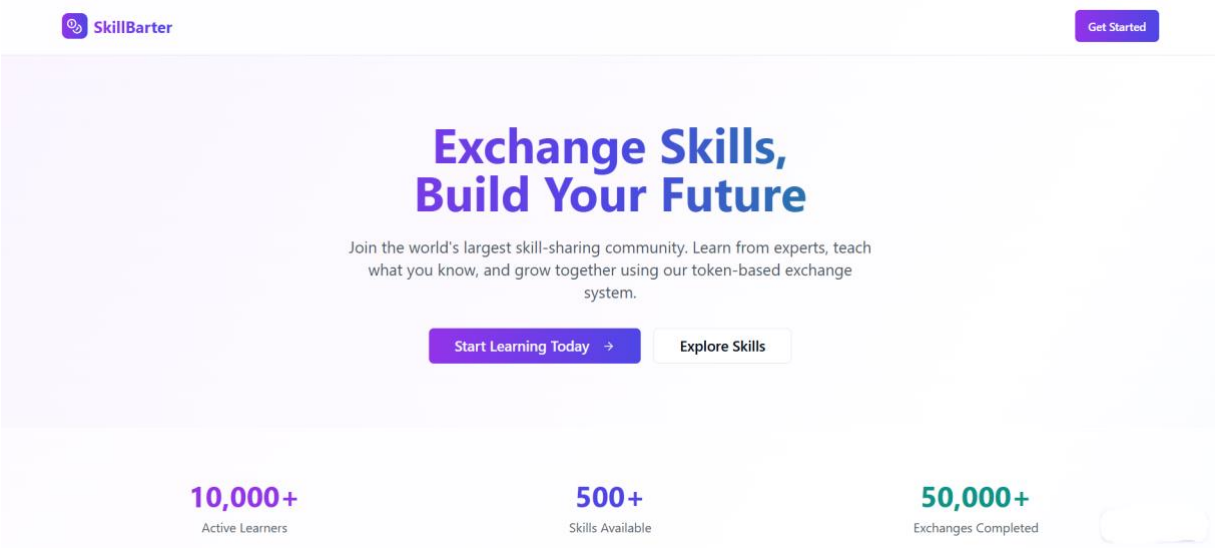
7. Modern Frontend Features

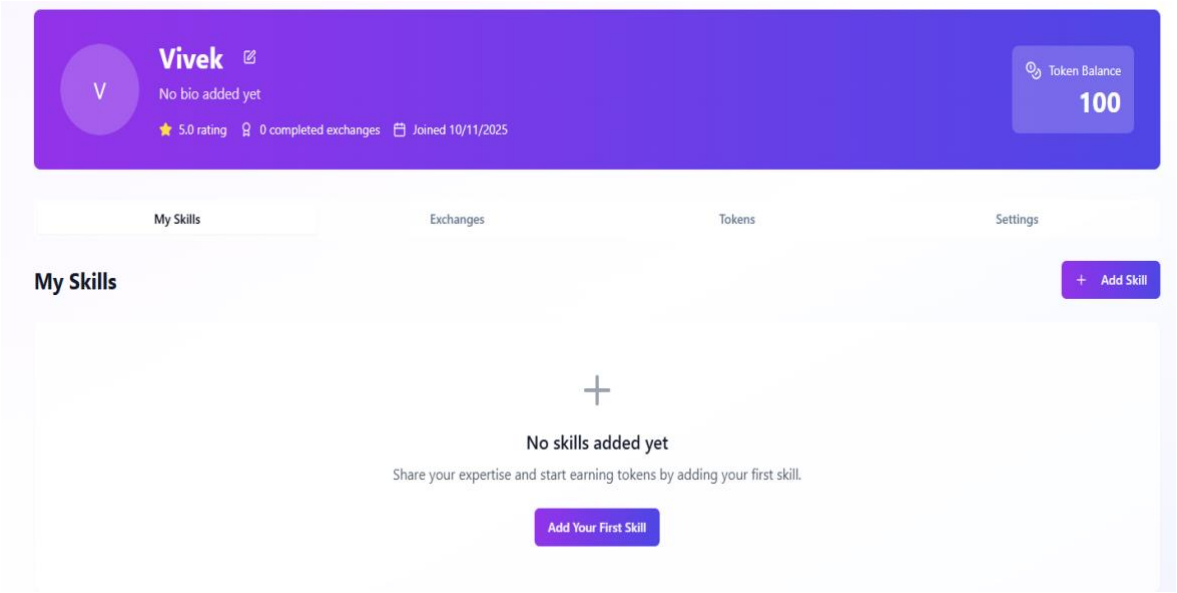
Custom Hooks and Context

```tsx
// Theme Context for dark/light mode
export const ThemeProvider = ({ children }: { children: React.ReactNode }) => {
  const [isDark, setIsDark] = useState(false);

  useEffect(() => {
    document.documentElement.classList.toggle('dark', isDark);
  }, [isDark]);

  return (
    <ThemeContext.Provider value={{ isDark, toggleTheme: () => setIsDark(!isDark) }}>
      {children}
    </ThemeContext.Provider>
  );
};

// Authentication Context
export const AuthProvider = ({ children }: { children: React.ReactNode }) => {
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [user, setUser] = useState(null);

  const login = async (credentials) => {
    const response = await authService.login(credentials);
```

```
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [user, setUser] = useState(null);

  const login = async (credentials) => {
    const response = await authService.login(credentials);
    setIsAuthenticated(true);
    setUser(response.user);
  };

  return (
    <AuthContext.Provider value={{ isAuthenticated, user, login }}>
      {children}
    </AuthContext.Provider>
  );
};
```

Output


SkillBarter
Get Started
Exchange Skills,
Build Your Future
Join the world's largest skill-sharing community. Learn from experts, teach what you know, and grow together using our token-based exchange system.
Start Learning Today
Explore Skills
10,000+
Active Learners
500+
Skills Available
50,000+
Exchanges Completed


V
Vivek
No bio added yet
5.0 rating
0 completed exchanges
Joined 10/11/2025
Token Balance
100
My Skills
Exchanges
Tokens
Settings
My Skills
Add Skill
No skills added yet
Share your expertise and start earning tokens by adding your first skill.
Add Your First Skill

V

Welcome back, Vivek!

Ready to learn something new today?

Token Balance	Active Exchanges	Completed	Rating
100	0	0	5.0 ★

Recent Activity

View All

No exchanges yet

Browse Skills

Quick Actions

- Browse Skills
- + Add Your Skill
- View Profile

Token Balance

100

Available Tokens

This Month Earned:	+50
This Month Spent:	-25

Featured Skills

View All ↗

JavaScript Development

25 tokens/hr

Programming

Learn modern JavaScript ES6+ features, async programming, and best practices

John Doe ★ 4.8

Intermediate · Expert

JavaScript · ES6 · Programming

Request Exchange

React Framework

30 tokens/hr

Programming

Master React hooks, state management, and component architecture

John Doe ★ 4.8

Advanced · Expert

React · Frontend · Components

Request Exchange

UI/UX Design Fundamentals

25 tokens/hr

Design

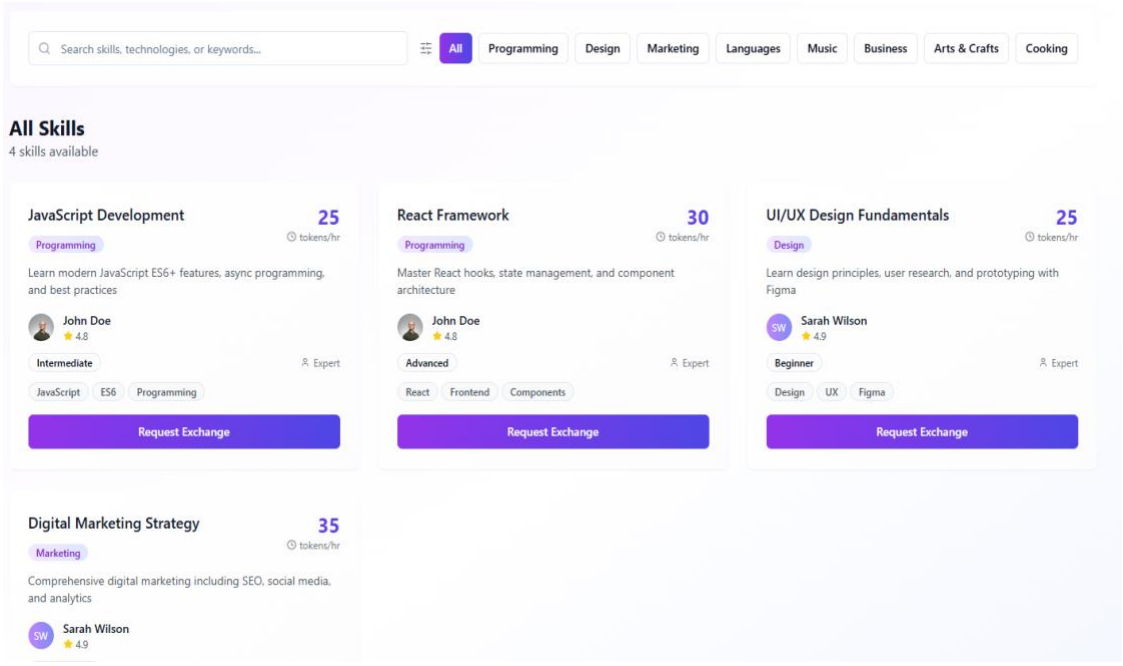
Learn design principles, user research, and prototyping with Figma

SW Sarah Wilson ★ 4.9

Beginner · Expert

Design · UX · Figma

Request Exchange



References

1. **React Documentation**: https://react.dev/
2. **TypeScript Handbook**: https://www.typescriptlang.org/docs/
3. **Tailwind CSS**: https://tailwindcss.com/docs
4. **Radix UI**: https://www.radix-ui.com/
5. **Vite Build Tool**: https://vitejs.dev/
6. **shadcn/ui Components**: https://ui.shadcn.com/
7. **Modern Web Development Practices**: Various online resources and documentation
8. **UX/UI Design Principles**: Material Design and Apple

Human Interface Guidelines

9. **Accessibility Guidelines**: WCAG 2.1 Standards
10. **Performance Optimization**: Web.dev performance guides